

实验二 组合部件的实现

一、实验目的

- 1. 学习运用硬件描述语言 Verilog 描述组合电路。
- 2. 学习使用 Quartus II 开发环境。
- 3. 理解多路复用器、译码器、运算单元的工作原理及实现方式。

二、实验原理

1. 多路复用器（多路选择器）

多路复用器从多个数据输入中选择一个，将其数据传输到输出。选择哪一个数据输入由一组选择输入控制。通常， 2^n 个数据输入需要 n 位选择输入，选择输入的位组合决定选择哪个数据输入。

8 重 3-1 多路复用器有 3 个数据输入、2 位选择输入、1 个输出，每个数据输入和输出都是 8 位，所以称之为 8 重，选择输入决定将哪个数据输入传至输出，8 重 3-1 多路复用器的端口及功能如下所示：

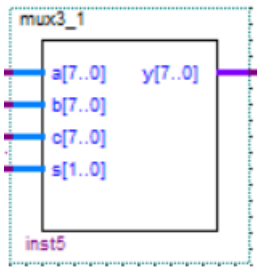


图 1 8 重 3-1 多路复用器的端口

表 1 8 重 3-1 多路复用器的端口介绍

端口名	I/O	描述
a[7:0]	Input	8 位数据输入 a
b[7:0]	Input	8 位数据输入 b
c[7:0]	Input	8 位数据输入 c
s[1:0]	Input	2 位选择输入
y[7:0]	Output	8 位数据输出

表 2 8 重 3-1 多路复用器的功能

S[1..0]	Y
00	a
01	b
10	c
其它组合	a

8 重 2-1 多路复用器有 2 个数据输入、1 个选择输入、1 个输出，每个数据输入和输出都是 8 位，选择输入决定将哪个数据输入传至输出，8 重 2-1 多路复用器的端口及功能如下所示：

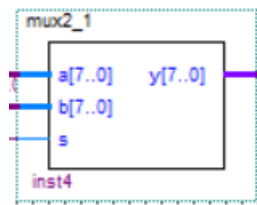


图 2 8 重 2-1 多路复用器的端口

表 3 8 重 2-1 多路复用器的端口介绍

端口名	I/O	描述
a[7:0]	Input	8 位数据输入 a
b[7:0]	Input	8 位数据输入 b
s	Input	1 位选择输入
y[7:0]	Output	8 位数据输出

表 4 8 重 2-1 多路复用器的功能

S	Y
0	a
1	b

2、指令译码器

译码是将一个 n 位的输入码转换成一个 m 位的输出码，并且 $n \leq m \leq 2^n$ ，以保证每一个有效的输入码都产生唯一的输出码。译码通过译码器实现，译码器是一个多输入、多输出的组合电路，输入端加载一个 n 位的二进制码，输出端产生一个对应的 m 位二进制码。

指令译码器根据指令系统表(表 5)中的机器码高 4 位，解释指令的操作码，判定是哪条指令，并置该指令对应的输出为 1，否则输出为 0。

表 5 指令系统表

汇编符号	功能	机器码	备注
MOVA Rd, Rs	$Rd \leftarrow (Rs)$	0100 Rd Rs	
MOVB M, Rs	$M(R0) \leftarrow (Rs)$	0101 00 Rs	
MOVC Rd, M	$Rd \leftarrow M((R0))$	0110 Rd 00	
MOVD R3, PC	$R3 \leftarrow PC$	0111 11 XX	
ADD Rd, Rs	$Rd \leftarrow (Rd) + (Rs)$	1000 Rd Rs	
SUB Rd, Rs	$Rd \leftarrow (Rd) - (Rs)$ IF $(Rd) > (Rs)$, THEN $G=1$, ELSE $G=0$	1001 Rd Rs	
JMP	$PC \leftarrow (R3)$	1010 XX 11	
JG	IF $G=1$, THEN $PC \leftarrow (R3)$	1011 XX 11	
IN Rd	$Rd \leftarrow$ 外设输入	1100 Rd XX	
OUT Rs	外设 $\leftarrow (Rs)$	1101 XX Rs	
MOVI #imm	$R0 \leftarrow \#imm$	1110 00 XX imm	双字节
HALT	停机	1111 00 00	

指令译码器的端口如下图所示，其中 **en** 为使能输入信号，**ir[3..0]**是指令的 4 位操作码（机器码的高 4 位），输出是对应的 12 条指令。指令译码器的功能如下表所示：

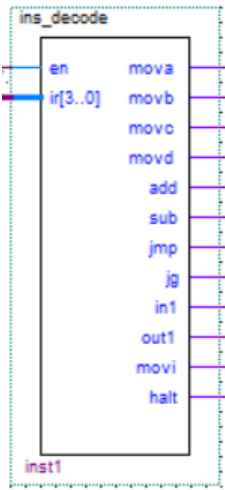


图 3 指令译码器的端口

表 6 指令译码器的功能

输入		输出											
en	ir[3:0]	mov _a	mov _b	mov _c	mov _d	add	sub	jmp	jg	in _l	out _l	mov _i	halt
0	xxxx	0	0	0	0	0	0	0	0	0	0	0	0
1	0100	1	0	0	0	0	0	0	0	0	0	0	0
1	0101	0	1	0	0	0	0	0	0	0	0	0	0
1	0110	0	0	1	0	0	0	0	0	0	0	0	0
1	0111	0	0	0	1	0	0	0	0	0	0	0	0
1	1000	0	0	0	0	1	0	0	0	0	0	0	0
1	1001	0	0	0	0	0	1	0	0	0	0	0	0
1	1010	0	0	0	0	0	0	1	0	0	0	0	0
1	1011	0	0	0	0	0	0	0	1	0	0	0	0
1	1100	0	0	0	0	0	0	0	0	1	0	0	0
1	1101	0	0	0	0	0	0	0	0	0	1	0	0
1	1110	0	0	0	0	0	0	0	0	0	0	1	0
1	1111	0	0	0	0	0	0	0	0	0	0	0	1
1	其它	0	0	0	0	0	0	0	0	0	0	0	0

3、算术单元 AU

运算器 ALU 是中央处理器（CPU）的重要组成部分，负责执行算术运算和逻辑运算。本模型机只有算术运算，算术单元 AU 实现 8 位二进制补码的加、减法运算。算术单元 AU 除了完成 ADD、SUB 运算外，还需在 MOVA、MOVB 和 OUT 三条指令执行时，将数据传送至总线 BUS。算术单元 AU 的端口及功能如下所示：

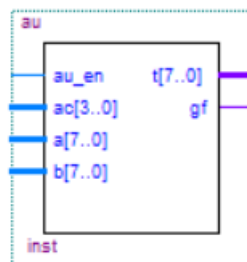


图 4 算术单元 AU 的端口

表 7 算术单元 AU 的端口介绍

端口名	I/O	描述
au_en	Input	使能输入信号
ac[3:0]	Input	控制信号，控制 AU 的操作
a[7:0]	Input	操作数，8 位补码
b[7:0]	Input	操作数，8 位补码
t[7:0]	Output	运算的结果，8 位补码
gf	Output	状态位，减法运算产生的状态位

表 8 算术单元 AU 的功能

au_en	ac[3..0]	t[7..0]	gf
1	1000	$t=a+b$	0
1	1001	$t=b-a$	IF(b>a), THEN gf=1, ELSE gf=0
1	0100、0101 或 1101	$t=a$	0
1	其它组合	$t=8'h00$	0
0	XXXX	$t=8'h00$	0

三、实验内容

1. 用 Verilog HDL 实现 8 重 2-1 多路复用器、8 重 3-1 多路复用器。
2. 用 Verilog HDL 实现指令译码器。
3. 用 Verilog HDL 实现运算单元 AU。

四、实验过程

1. 基于 Quartus II 创建工程，采用 VERILOG 语言分别描述多路复用器、指令译码器和算术单元 AU。
2. 编译（compilation）通过后，基于 Quartus II 进行仿真验证，测试电路的正确性。
3. 在 Tool 下用 netlist viewer 查看 RTL viewer，了解语句描述对应的 RTL 视图。
4. 仿真验证正确后，将.v 文件压缩为.zip，提交至评测平台测试。
5. 撰写实验报告，将报告和工程文件提交至学习通平台的实验作业中。

五、思考题

1. 采用 always 语句描述时序逻辑电路时，应该采用阻塞赋值语句还是非阻塞赋值语句？为什么？
2. Verilog HDL 运用 IF 和 CASE 语句描述组合电路时，如何确保描述的电路没有综合出锁存器？
3. 运用 always 语句描述组合部件时，应该采用阻塞赋值还是非阻塞赋值？为什么？